Database Systems

Piya Limcharoen

Version 2026/1

Contents

Chapter 5: Column-Family and Graph Databases 6
1.1 Chapter Overview . . . 6
1.2 Column-Family (Wide-Column) Stores . . . 8
1.2.1 A Normal Relational Table (RDBMS) . . . 8
1.2.2 The Problem with Normal Tables at Scale . . . 9
1.2.3 The Transformation: Changing to a Column-Family Store . . . 10
1.2.4 Visualizing the Transformed Structure . . . 11
1.2.5 Mental Model: A Dictionary of Dictionaries . . . 12
1.2.6 Cell Identification: The Four-Coordinate Addressing Model . . . 13
1.2.7 Comparison: Relational Table vs. Column-Family Store . . . 14
1.2.8 Use Case: High-Velocity IoT Sensor Data . . . 15
1.3 Storage Engine: LSM Trees (Log-Structured Merge Trees) . . . 16
1.3.1 How LSM Trees Work (The Write Path) . . . 17

1.3.2 Write-Ahead Log (WAL): RDBMS vs. LSM Tree . . . 21
1.3.3 Storage Engine Comparison: B+ Tree vs. LSM Tree . . . 22
1.4 Column-Family Tool: Apache Cassandra . . . 24
1.4.1 Cassandra Data Organization . . . 26
1.5 Cassandra Query Language (CQL) . . . 30
1.5.1 Primary Key: The Most Important Concept . . . 31
1.5.2 Example: Physical Data Layout and Query Rules . . . 32
1.6 Graph Databases . . . 33
1.6.1 Why Do We Need Graph Databases? The Join Explosion Problem . . . 37
1.6.2 The Property Graph Model . . . 39
1.6.3 Visualizing a Property Graph . . . 41
1.6.4 Index-Free Adjacency (The Key Advantage) . . . 42
1.6.5 Real-World Graph Use Cases . . . 43
1.7 Graph Database Tool: Neo4j . . . 44
1.8 Graph Query Language: Cypher . . . 45
1.8.1 Fundamental Query Patterns: Direction & Isolated Nodes . . . 46
1.8.2 1. CREATE and MERGE (Insert Data) . . . 47
1.8.3 2. MATCH and WHERE (Read and Filter Data) . . . 48
1.8.4 3. SET and DELETE (Update and Remove Data) . . . 49
1.8.5 4. Advanced: Variable-Length Paths and Aggregation . . . 50
1.8.6 Using Neo4j with Python . . . 51
1.8.7 When NOT to Use Graph Databases . . . 53
1.9 Chapter Summary . . . 54
1.9.1 SQL and NoSQL Design choices: Multi-Database (Polyglot) Architecture . . . 55
1.9.2 SQL and NoSQL Databases Short Summary . . . 57
1.10 Activity/Homework: Graph Database Exploration (Neo4j) . . . 58
1.10.1 Expected Learning Outcomes . . . 58

1.10.2 Technology Stack 59
1.10.3 The Movie Graph Data Model & Schema 60
1.10.4 Environment Setup Instructions 61
1.10.5 Student Tasks: 30 Cypher Queries 63
1.10.6 Deliverables 66

Copyright Notice

Copyright © 2026 Piya Limcharoen. All rights reserved.

This lecture note was authored by Piya Limcharoen. Artificial Intelligence (AI) was utilized as a tool for language enhancement and grammatical correction; however, all intellectual property, original concepts, and the structure of this work belong exclusively to the author.

Terms of Use

This document is provided for educational purposes only and is intended for the exclusive use of students currently enrolled in this course.

The reproduction, distribution, public display, or transmission of this material, in whole or in part, in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—is strictly prohibited without the prior written permission of the author. Students may not share, upload, or distribute these notes outside of the official class environment.

Warning

Any unauthorized use of this material will be considered academic misconduct and will be subject to university disciplinary procedures. Furthermore, such an act may constitute an infringement of copyright law and is subject to legal action.

Chapter 5:
Column-Family and Graph Databases

Chapter 5: Column-Family and Graph Databases

1.1 Chapter Overview

In previous chapters, we studied Relational databases (Chapters 1–2), Document databases (MongoDB, Chapter 3), and Key-Value stores (Redis, Chapter 4).

In this chapter, we explore the remaining two NoSQL models, each designed for high performance in specialized domains:

1. **Column-Family Stores (e.g., Apache Cassandra):**
   Built for **massive scalability and high-velocity writes**. These systems are used by giants like Netflix and Apple to handle petabytes of data across thousands of servers. We will explore how their unique storage engine, the **LSM Tree (Log-Structured Merge Tree)**, achieves extreme write speeds using sequential disk I/O.

2. **Graph Databases (e.g., Neo4j):**
   Built for **complex relationship traversal**. While relational databases slow down drastically with multi-table JOINs (the "Join Explosion" problem), graph databases excel at finding patterns across connected data using **Index-Free Adjacency** and the **Cypher** query language.

Learning Objectives

By the end of this chapter, you will be able to:

- **Understand Column-Family Stores:** Explain the 4-coordinate cell addressing model: Row Key, Column Family, Column (Qualifier), Timestamp.
- **Understand LSM Trees:** Contrast B+ Trees (random in-place updates) with LSM Trees (fast sequential append-only writes: WAL, MemTable, SSTable, Compaction).
- **Design Cassandra Schemas (CQL):** Use **Partition Keys** (data distribution) and **Clustering Keys** (on-disk sorting), and recognize query constraints.
- **Explain Index-Free Adjacency:** Understand why graph traversal is $O(k)$ based on local connections rather than total database size.
- **Model Data with Property Graphs:** Identify Nodes, Labels, Directed Relationships, and Properties.
- **Write Cypher Queries:** Use ASCII-art syntax in Neo4j to `MATCH`, `WHERE`, `CREATE`, `MERGE`, and traverse relationships.
- **Select the Right Database:** Apply Multi-Database (Polyglot) thinking to match database types to application workloads.

1.2 Column-Family (Wide-Column) Stores

A **Column-Family** database (also called a Wide-Column store) is a distributed NoSQL database designed for massive data volumes and high write throughput across many cluster nodes.

1.2.1 A Normal Relational Table (RDBMS)

In a traditional relational database, data is stored in a rigid two-dimensional table where **every row must share the exact same predefined columns**:

user_id	name	email	age	last_login	ip_address	theme	loyalty
U101	Alice	alice@ku.th	25	10:15	192.168.1.1	*NULL*	*NULL*
U205	Bob	bob@ku.th	*NULL*	*NULL*	*NULL*	Dark	*NULL*
U308	Charlie	charlie@ku.th	31	*NULL*	*NULL*	*NULL*	450
U412	Diana	*NULL*	*NULL*	22:40	10.0.0.12	Light	1200

Table 1: A Relational Table with Sparse Attributes and Explicit NULLs

- **Rigid Schema:** All 8 columns are allocated for every row, even if only 2 or 3 are actually used.
- **Sparse (not dense) Data:** Missing attributes must be explicitly represented as *NULL*.

1.2.2 The Problem with Normal Tables at Scale

When relational tables scale to hundreds of millions of rows and hundreds of optional features, this rigid structure creates three major operational problems:

- **1. Wasted Space from Sparsity:**
  In modern applications (e.g., user profiles, IoT sensors, product catalogs), an entity might only use 10 out of 1,000 available attributes. Storing thousands of `NULL` placeholders or column offset headers wastes significant storage and memory bandwidth.

- **2. The `ALTER TABLE` Schema Bottleneck:**
  Adding a new attribute requires running `ALTER TABLE`. On multi-gigabyte or terabyte tables distributed across servers, schema migration locks tables, takes hours or days, and risks application downtime.

- **3. Heterogeneous (made up of parts) Entities:**
  Different users have different attributes (e.g., standard users vs. social login users vs. IoT devices). Relational tables force every entity into the same rigid rectangular mold.

1.2.3 The Transformation: Changing to a Column-Family Store

A **Column-Family store** solves these limitations by reorganizing data into logical groups called **Column Families** and only storing non-null attributes:

Step 1: Group Related Columns into Column Families

Columns that are queried together are grouped into a logical container called a **Column Family** (stored together on disk):

- **PersonalInfo**: `name`, `email`, `age`
- **ActivityLog**: `last_login`, `ip_address`
- **Preferences**: `theme`, `loyalty`

Step 2: Dynamic Columns (Zero NULLs Stored)

- Each row key only stores the columns that **actually contain data**.
- `NULL` values are **never stored** on disk — consuming **zero bytes**!
- Any row can add new columns on-the-fly at any time without schema migration.

1.2.4 Visualizing the Transformed Structure

Here is how the exact same four user records look after transforming into a Column-Family store:

Row Key	Family: PersonalInfo	Family: ActivityLog	Family: Preferences
U101	name: "Alice" email: "alice@ku.th" age: 25	last_login: "10:15" ip: "192.168.1.1"	*(no data stored)*
U205	name: "Bob" email: "bob@ku.th"	*(no data stored)*	theme: "Dark"
U308	name: "Charlie" email: "charlie@ku.th" age: 31	*(no data stored)*	loyalty: 450
U412	*(no data stored)*	last_login: "22:40" ip: "10.0.0.12"	theme: "Light" loyalty: 1200

Table 2: Column-Family View: Dynamic Attributes Grouped by Family

- Each row has its own dynamic set of columns under each family.
- If a row has no data for a column family or attribute, **nothing is stored**.

1.2.5 Mental Model: A Dictionary of Dictionaries

In programming terms, think of a Column-Family store not as a 2D matrix, but as a **nested hash map** (a dictionary of dictionaries):

Listing 1: Two-Level Map Structure of Column Families

```
RowKey: "U101" -> {
    "PersonalInfo": { "name": "Alice", "email": "alice@ku.th", "age": "25" },
    "ActivityLog":  { "last_login": "10:15", "ip": "192.168.1.1" }
}

RowKey: "U205" -> {
    "PersonalInfo": { "name": "Bob", "email": "bob@ku.th" },
    "Preferences":  { "theme": "Dark" }
}

RowKey: "U308" -> {
    "PersonalInfo": { "name": "Charlie", "email": "charlie@ku.th", "age": "31" },
    "Preferences":  { "loyalty": "450" }
}
```

- **Level 1 (Row Key):** Locates the row on the distributed cluster (e.g., `"U101"`).
- **Level 2 (Column Family):** Locates the file/storage directory on disk (e.g., `"PersonalInfo"`).
- **Level 3 (Column Qualifier):** Locates the dynamic key-value pair (e.g., `"email"` → `"alice@ku.th"`).

1.2.6 Cell Identification: The Four-Coordinate Addressing Model

In a Column-Family database, every individual piece of data (called a **cell**) is uniquely located using four coordinates:

$$(\text{Row Key}, \text{Column Family}, \text{Column Qualifier}, \text{Timestamp}) \longrightarrow \text{Value} \tag{1}$$

Example:

$$(\texttt{User_101}, \texttt{PersonalInfo}, \texttt{Name}, \texttt{2026-11-02}) \longrightarrow \texttt{"Alice"}$$

Coordinate	Meaning and Purpose
1. Row Key	Uniquely identifies each row (e.g., `User_101`). Data is partitioned and sorted by this key.
2. Column Family	A logical group of related columns stored together on disk (e.g., `PersonalInfo`). Defined in schema.
3. Column Qualifier	The actual column name inside the family (e.g., `name`, `age`). Can be added dynamically.
4. Timestamp	Attached to every cell. Allows storing multiple versions of the same data and resolving write conflicts.

Table 3: The Four Coordinates Defining a Cell

1.2.7 Comparison: Relational Table vs. Column-Family Store

Dimension	**Relational Table (RDBMS)**	**Column-Family Store (NoSQL)**
Structure	Rigid 2D table (fixed rows & columns)	Row Key → Column Family → Column Qualifier → Timestamp
Schema	Fixed; all rows share identical columns	Flexible; each row has dynamic columns
Missing Values	Explicit `NULL` stored	Non-existent cells; zero bytes stored
Adding Columns	Heavy `ALTER TABLE` operation	Dynamic write without schema changes
Physical Storage	Row-oriented (entire row stored together)	Column-family oriented (families stored in separate files)
Best For	Complex relational queries, ACID	Massive writes, sparse data, distributed scale

Table 4: Relational Table vs. Column-Family Store

1.2.8 Use Case: High-Velocity IoT Sensor Data

Column-Family stores are the industry standard for **Time-Series Data** and **Internet of Things (IoT)**.

Scenario: Smart Traffic Monitoring

- 1,000,000 traffic sensors across a city send status updates every second.
- **Workload:** 1 million writes per second, continuous ingestion.
- **Query:** "Show me the speed readings of Sensor #500 for the last hour."

Why RDBMS Struggles:

- Trying to insert 1M rows/second into MySQL causes massive lock contention on the B+ Tree index.
- Disk heads thrash with random writes, causing the database to freeze.

Why Column-Family Wins:

- **Instant Ingestion:** Writes hit the in-memory MemTable immediately.
- **Sequential Storage:** Flushes to disk sequentially without random seek delays.
- **High Compression:** Because columns store similar data types (e.g., float speeds or timestamps), compression ratios are very high, drastically saving disk space.

1.3 Storage Engine: LSM Trees (Log-Structured Merge Trees)

Traditional relational databases use **B+ Trees** (the B-Tree variant studied in Chapter 2), which are optimized for **Reads** but become slow for **Writes** due to random disk I/O (random page access).

Column-Family stores (like Cassandra and HBase) use **LSM Trees**, designed specifically for **Massive Write Throughput**.

Why Fast for Writes: Sequential Append-Only I/O

Writing a continuous stream of data sequentially to disk is **orders of magnitude faster** than jumping around to update different random blocks. LSM Trees achieve extreme speed by turning random writes into fast sequential writes.

Example: High-Velocity IoT Sensor Ingestion

- **100,000 Datapoints/Sec:** Imagine 100,000 IoT sensors (smart meters, weather stations, vehicles) each reporting telemetry once per second. That generates **100,000 new records every second**.
- **Why B+ Trees overload:** Records arrive with different sensor IDs, forcing the DBMS to seek random leaf pages scattered across disk, update pages in-place, and split nodes—causing severe disk thrashing.
- **Why LSM Trees handle:** Data points are simply appended sequentially to a commit log on disk and absorbed into memory (**MemTable**)—zero random seeking, effortlessly handling massive ingestion.

1.3.1 How LSM Trees Work (The Write Path)

LSM Trees achieve high performance using an **append-only** strategy in four steps:

1. **Write-Ahead Log (WAL / CommitLog):**
   Every write is first appended sequentially to a log file on disk. If the server crashes, data can be recovered by replaying this log.

2. **MemTable (Memory Table):**
   The write is then inserted into the **MemTable**—a sorted in-memory structure (RAM). Because writes happen in RAM, response time is instant (< 1 ms).

3. **Flushing to SSTable (Sorted String Table):**
   When the MemTable fills up, it is frozen and flushed sequentially to disk as a new **SSTable**.

   - **Sorted:** The data on disk is already sorted by key.
   - **Immutable:** SSTables are **never modified**. Updates or deletes simply create new versions in newer SSTables.

4. **Compaction:**
   To prevent having too many SSTable files, a background process merges multiple SSTables together, keeps only the latest timestamp for each cell, and removes deleted records.

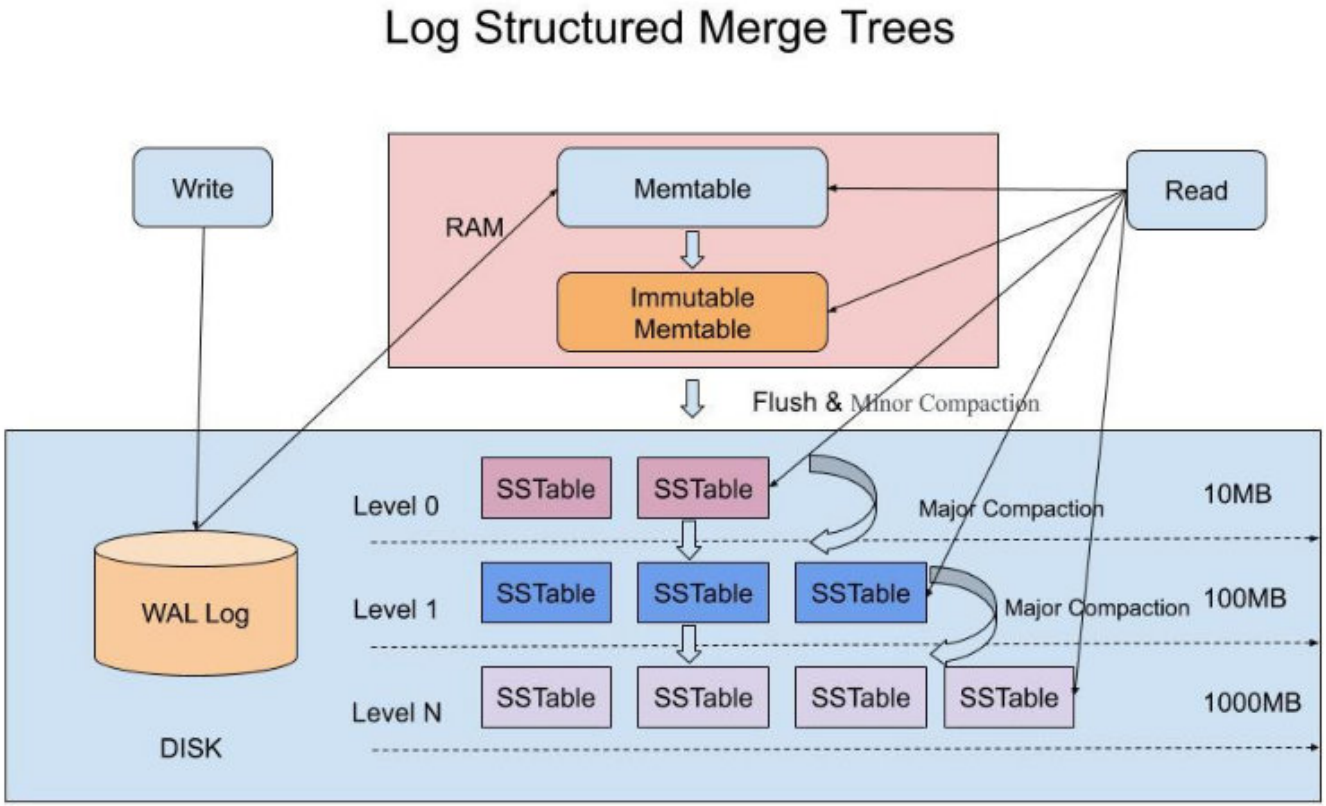


Figure 1: Architecture of a Log-Structured Merge Tree (LSM Tree)

Source: The Power of LSM-Tree by Muhamed Abdalla

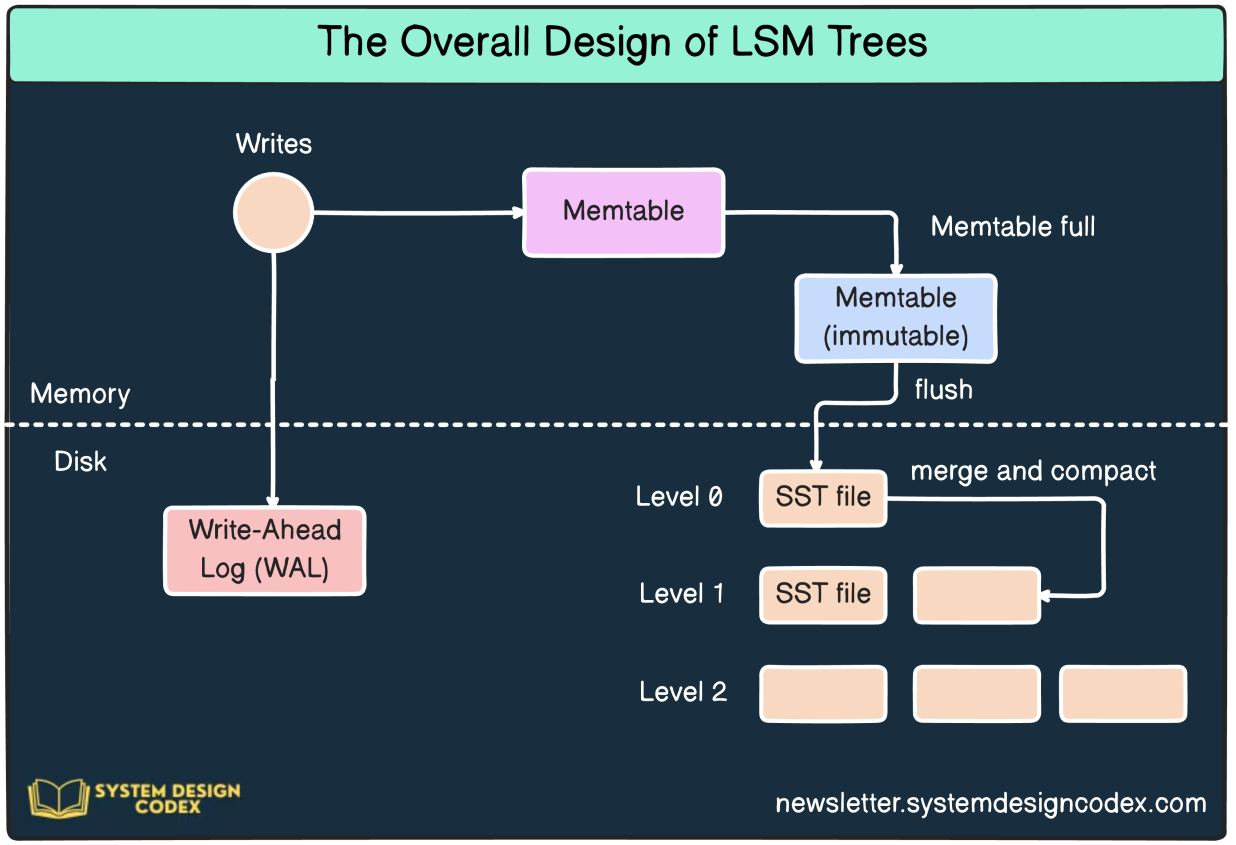


Figure 2: LSM-Tree Components: WAL, MemTable, and SSTables

Source: An Intro to LSM-Trees by System Design Codex

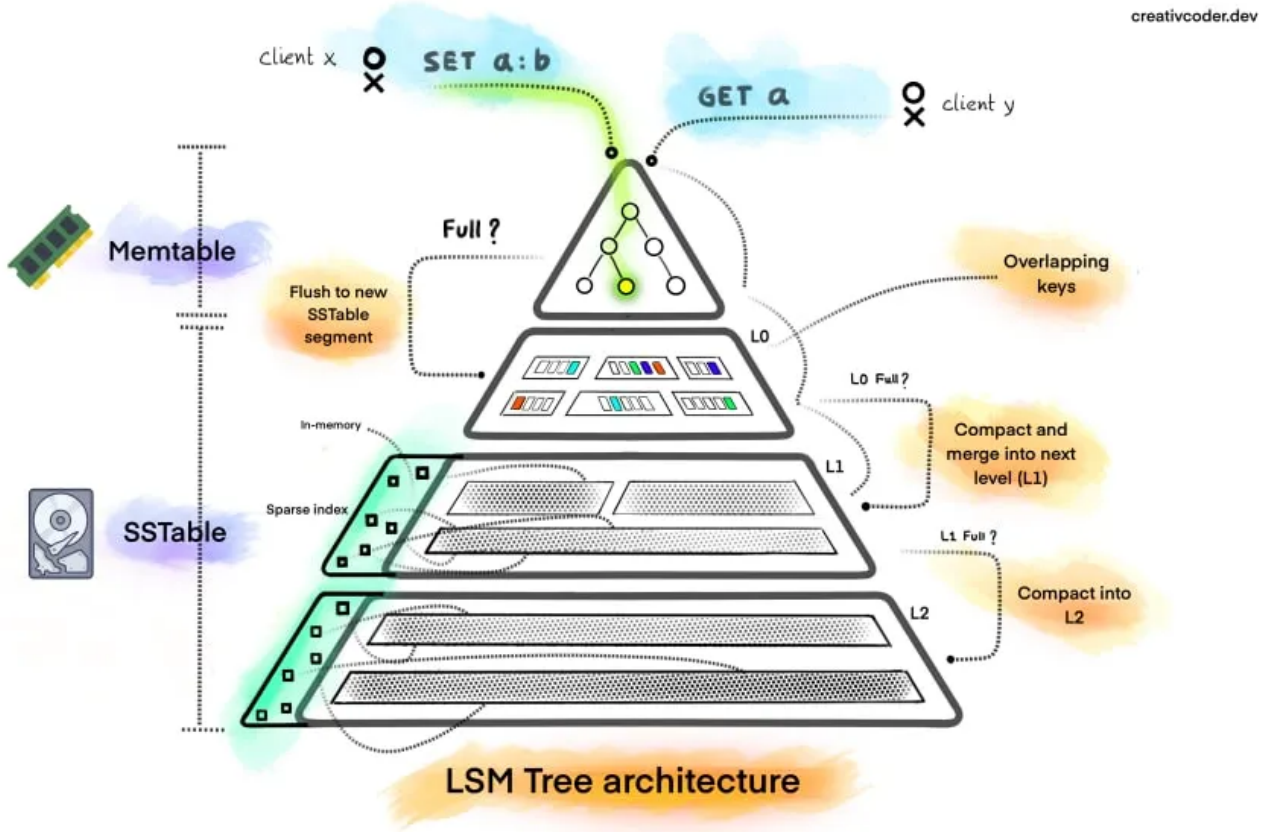


Figure 3: LSM-Tree Write Path and Compaction Process

Source: Secret Sauce Behind NoSQL LSM Tree by InterviewNoodle

1.3.2 Write-Ahead Log (WAL): RDBMS vs. LSM Tree

Recall from Chapters 1 and 2 that relational databases rely on Write-Ahead Logging. Column-Family NoSQL systems use the exact same concept for crash durability, but with a fundamental difference in how data reaches persistent storage:

Dimension	**RDBMS (PostgreSQL, SQLite, MySQL)**	**LSM Tree (Cassandra, HBase, RocksDB)**
Exact Name	**WAL** (Postgres, SQLite) **Redo Log** (MySQL, Oracle)	**WAL** (HBase, RocksDB) **CommitLog** (Cassandra)
Why "Write-Ahead"?	Written to disk **ahead of** dirty buffer pool pages reaching data files	Appended to disk **ahead of** acknowledging writes stored in volatile RAM (**MemTable**)
Core Purpose	**Durability & Crash Recovery**: Replaying log restores committed transactions	**Durability & Crash Recovery**: Replaying log fully reconstructs the in-memory MemTable
Disk I/O Pattern	Sequential append-only I/O	Sequential append-only I/O
Later write operations	**Random I/O**: Must eventually write modified pages in-place into B+ Tree data files	**Sequential I/O**: MemTable flushes as a new, immutable **SSTable** (zero random seeks)

Table 5: Write-Ahead Log (WAL): RDBMS vs. LSM Tree Comparison

The Key Architectural Difference

In an RDBMS, the WAL is fast and sequential, but the engine is still chained to **random in-place disk I/O** for its B+ Tree. In an LSM Tree, the WAL is the **only immediate disk write**—all disk writes remain 100% sequential!

1.3.3 Storage Engine Comparison: B+ Tree vs. LSM Tree

Traditional relational databases prioritize **instant reads**, while Column-Family LSM storage engines prioritize **extreme write velocity**:

Feature	**B+ Tree (RDBMS)**	**LSM Tree (Column-Family NoSQL)**
Optimized For	Read-heavy workloads	**Write-heavy workloads**
Disk Access	Random I/O (Update in-place)	**Sequential I/O** (Append-only)
Write Speed	Slower (Locking & page splits)	**Extremely Fast** (RAM + sequential log)
Read Speed	Very Fast (Direct single-page read)	Slightly slower (Checks RAM + disk files)

Table 6: Storage Engine Comparison: B+ Tree vs. LSM Tree

Decision the architecture

If your workload consists of continuous, high-velocity data ingestion (e.g., IoT sensors, application logs, financial tick feeds), an **LSM Tree**-based database is vastly superior to a traditional B+ Tree relational database.

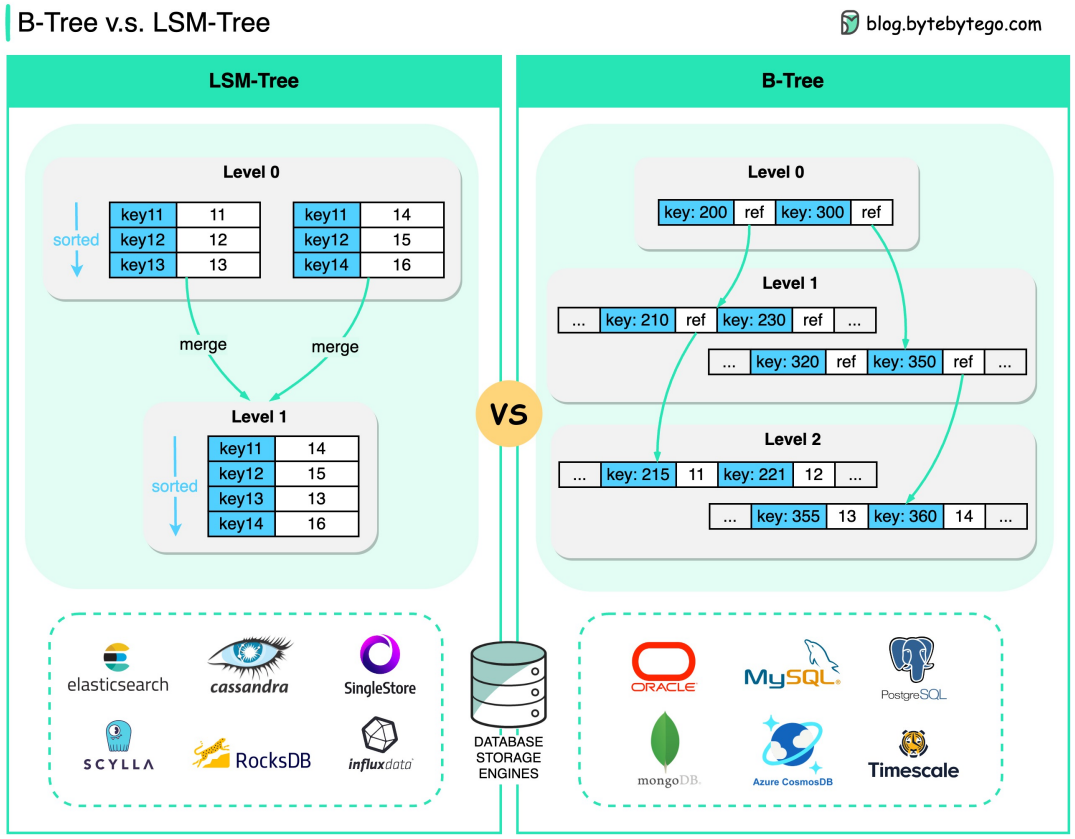


Figure 4: B+ Tree (In-Place Update) vs. LSM-Tree (Append-Only Sequential)

Source: ByteByteGo - B-Tree vs. LSM-Tree

1.4 Column-Family Tool: Apache Cassandra

Apache Cassandra is an open-source distributed wide-column store designed to handle massive amounts of data across many commodity servers.

Key Architectural Features:

- **Decentralized (Masterless):**
  Every node in the cluster is identical (peer-to-peer). There is no "Master" node, meaning there is **no Single Point of Failure (SPOF)**.

- **Linear Scalability:**
  Performance increases predictably as you add more nodes. If 2 nodes handle 100k writes/sec, 4 nodes will handle 200k writes/sec.

- **Tunable Consistency:**
  Developers can choose the trade-off between consistency and availability ($R + W > N$) for each query.

- **Multi-Data Center Replication:**
  Data can be replicated across geographical regions automatically.

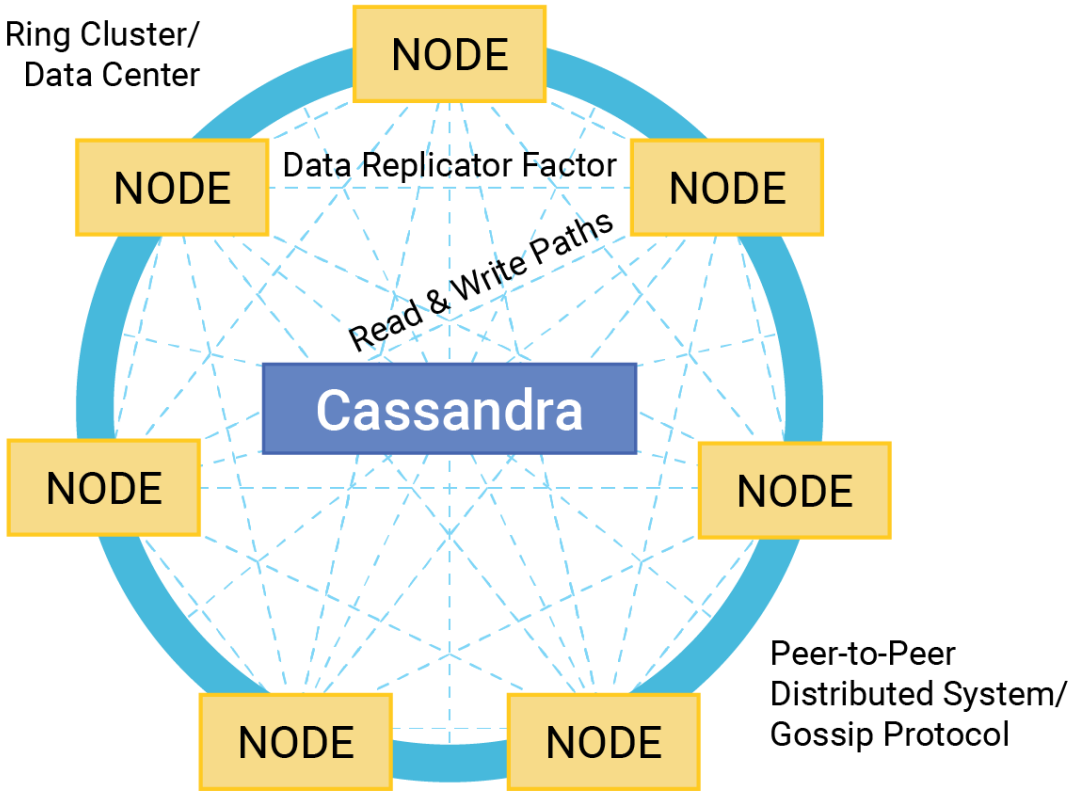


Figure 5: Apache Cassandra Masterless Ring Architecture

Source: ScyllaDB - Apache Cassandra Architecture

1.4.1 Cassandra Data Organization

Although Cassandra is a wide-column (column-family) database, it organizes data using tables with predefined columns rather than explicitly grouping columns into named Column Families as in Bigtable/HBase.

- **Keyspace:** A namespace (similar to a database in SQL).
  - **Table:** A collection of related data with predefined columns.
    - **Partition Key:** Determines which partition stores the data and how it is distributed across nodes.
      - **Clustering Key:** Determines the ordering of rows within the same partition.

Key Difference

A table is divided into partitions, each containing rows with the same Partition Key.

- **Partition Key:**
  One or more columns that determine how rows are grouped into partitions and distributed across nodes.
- **Clustering Key:**
  Orders rows within a partition.

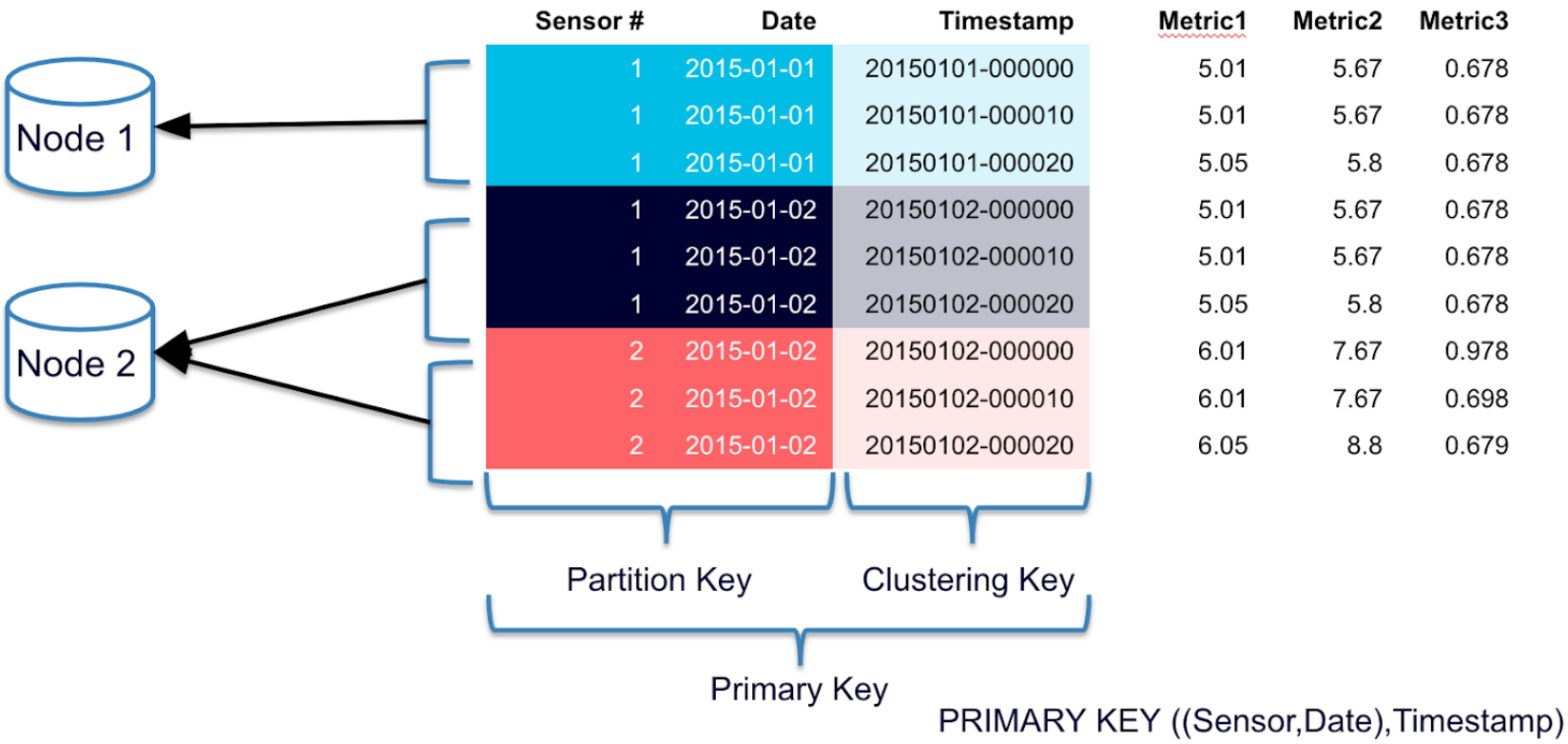


Figure 6: Cassandra Partitioning: Partition Key vs. Clustering Key

Source: Instaclustr - Cassandra Data Partitioning

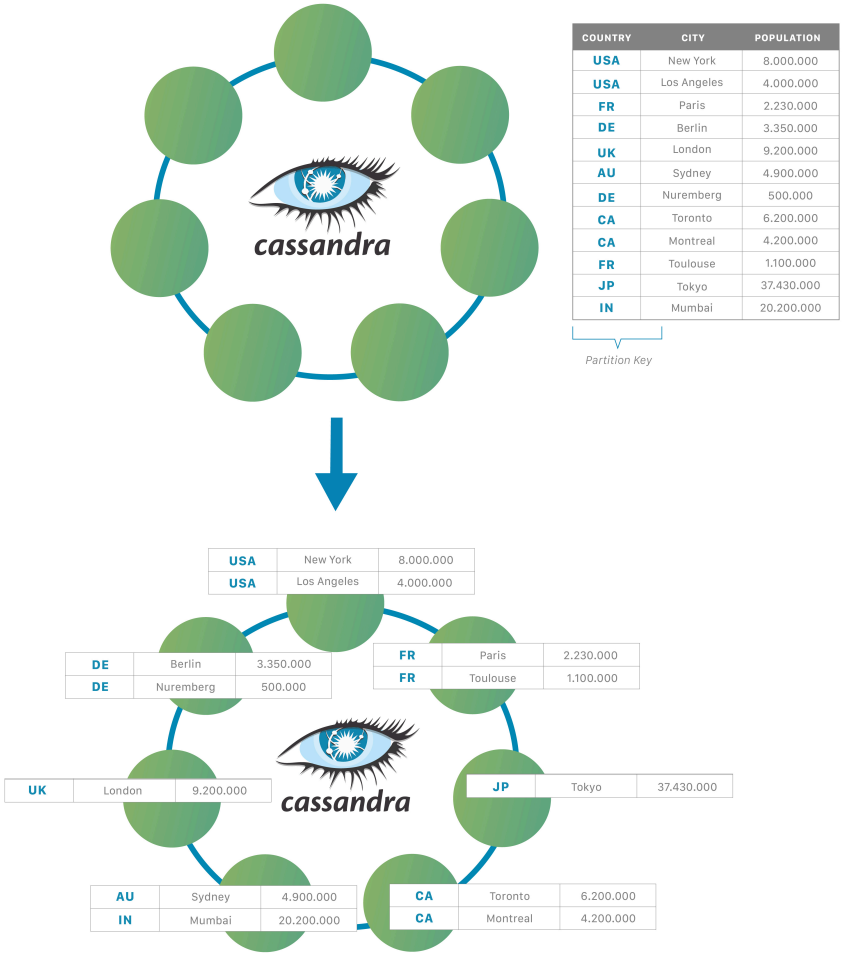


Figure 7: Apache Cassandra: Dynamic Scaling and Data Distribution

Source: Apache Cassandra Documentation: Cassandra Basics

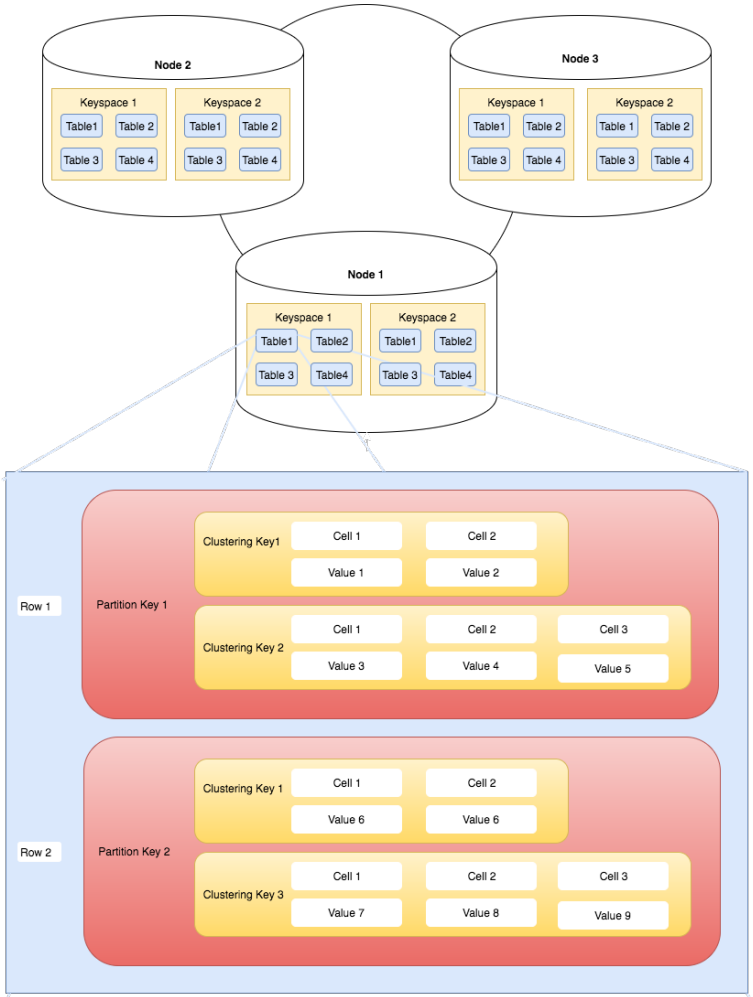


Figure 8: Cassandra Data Hierarchy: Cluster, Keyspace, Table, and Partitions

Source: A Bias For Action - Apache Cassandra CQL Tutorial

1.5 Cassandra Query Language (CQL)

To interact with Cassandra, we use **CQL (Cassandra Query Language)**.

CQL looks very similar to SQL (`SELECT`, `INSERT`, `UPDATE`, `DELETE`), which makes it feel familiar. However, the database behaves completely differently under the hood:

- **No JOIN Operations:**
  Data must be pre-joined (denormalized) before storing. **CQL has no JOIN support whatsoever**, because joining tables across distributed servers over a network destroys performance.

- **Strict WHERE Clause Requirements:**
  You **must** query using the Primary Key. Querying by an unindexed column is rejected by default to prevent full-cluster scans.

- **No Complex Aggregations:**
  Operations like `GROUP BY` or `SUM()` over entire tables are avoided.

1.5.1 Primary Key: The Most Important Concept

In Cassandra, you design tables based on **how you query the data**:

$$\textbf{Primary Key} = \textbf{Partition Key} + \textbf{Clustering Key(s)} \tag{2}$$

Listing 2: CQL Table Definition

```
CREATE TABLE traffic_data (
    sensor_id UUID,
    recorded_at TIMESTAMP,
    cars_passed INT,
    avg_speed FLOAT,
    PRIMARY KEY ((sensor_id), recorded_at)
) WITH CLUSTERING ORDER BY (recorded_at DESC);
```

- **Partition Key (`sensor_id`):**
  Decides **which node** stores the data.
- **Clustering Key (`recorded_at`):**
  Sorts the data **inside that server** on disk (newest first).

1.5.2 Example: Physical Data Layout and Query Rules

Listing 3: Sample Stored Partition Data

```
Partition: sensor_id = 'S1' (Stored together on Server 1)
-----------------------------------------------------------
recorded_at (Clustering Key) | cars_passed | avg_speed
-----------------------------------------------------------
10:00 (Newest)               | 15          | 52.3
09:55                        | 18          | 49.8
09:50                        | 12          | 61.2
```

Valid Query (Efficient):

```
SELECT * FROM traffic_data
WHERE sensor_id = 'S1' AND recorded_at > '09:50';
```

Why valid? The Partition Key is provided. Cassandra knows exactly which server to read, and reads the sorted slice in one quick seek.

Invalid Query (The query is rejected before execution):

```
SELECT * FROM traffic_data WHERE avg_speed > 60;
```

Why rejected? No Partition Key is given. Cassandra would need to scan every server in the entire cluster, which degrades performance.

1.6 Graph Databases

A **Graph Database** is a NoSQL database designed for data where the **relationships between entities** are just as important as the entities themselves.

While relational databases focus on tables and rows, graph databases focus on:

Nodes + Relationships + Properties

Core Philosophy

In relational databases, relationships are computed at query time using JOIN.
In graph databases, relationships are stored directly as first-class citizens.

Ideal Use Cases:

- **Social Networks:** Friends-of-friends, mutual connections, follower feeds.
- **Fraud Detection:** Uncovering money laundering rings and circular transfer loops.
- **Recommendation Systems:** "People who bought X also bought Y."
- **Knowledge Graphs:** Connecting concepts, semantic search, and AI reasoning.

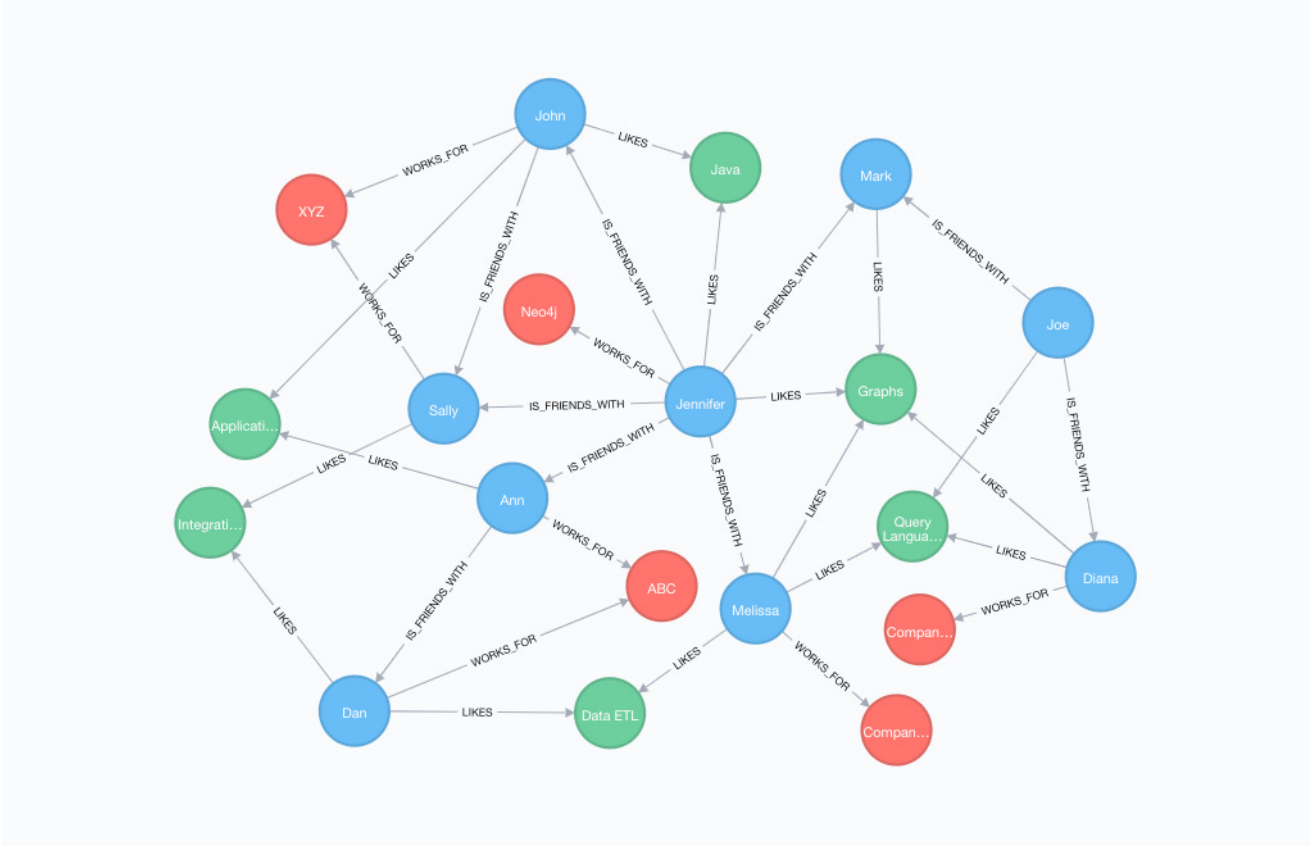


Figure 9: NoSQL Comparison: SQL vs. Graph Database Models

Source: Software Engineering WK - NoSQL vs SQL vs Graph Database Types

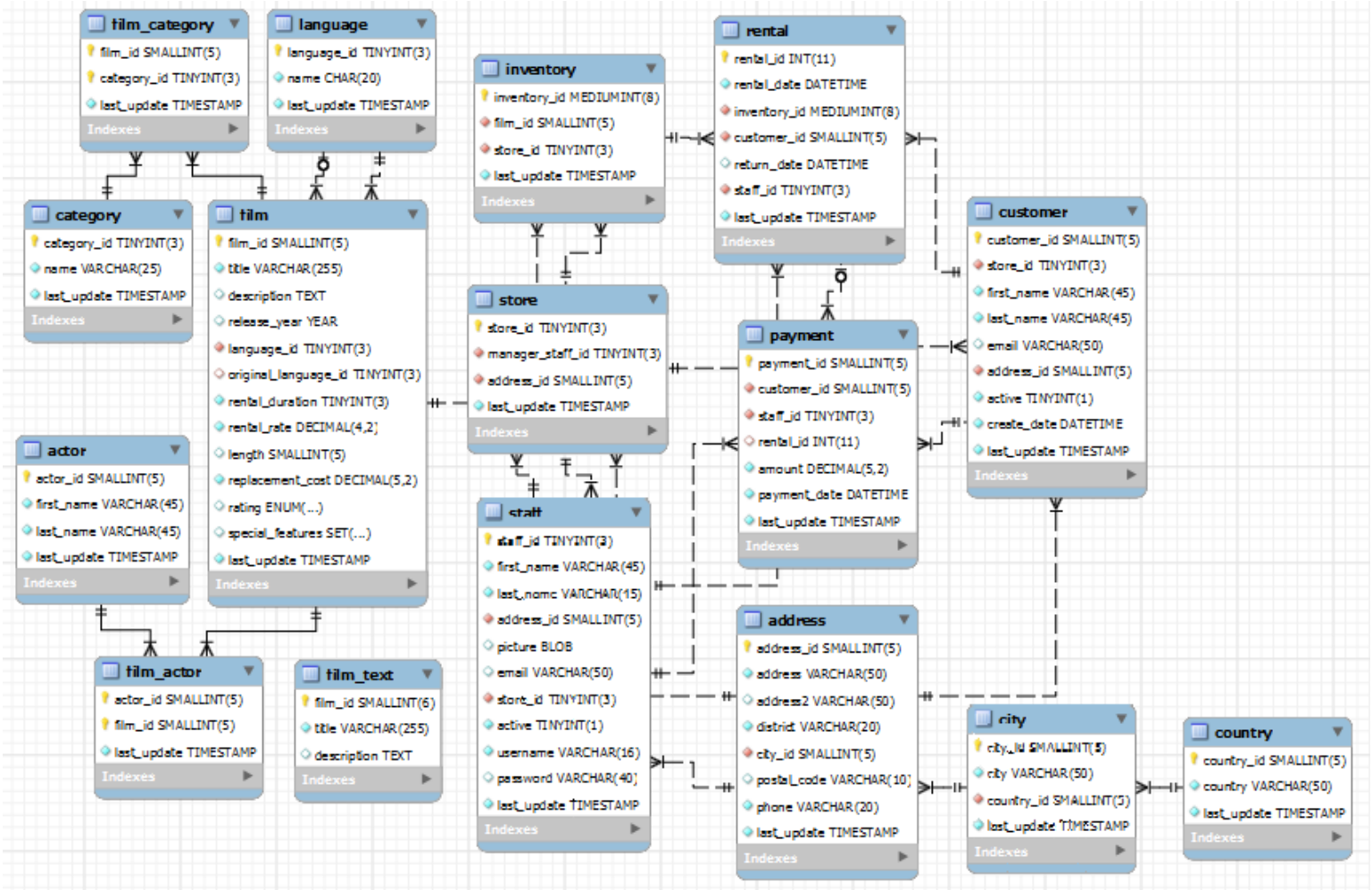


Figure 10: Relational Schema Complexity and Multi-Table JOINs: MySQL Sakila Sample Database

Source: NTU - MySQL Sample Databases (Sakila DVD Rental)



Figure 11: Relational Schema Complexity: Point of Sale (POS) Entity-Relationship Diagram

Source: DataModelPack - Point of Sale ERD

1.6.1 Why Do We Need Graph Databases? The Join Explosion Problem

Relational databases work well for structured business records. But they struggle when relationships are deep or recursive:

The Problem: "Find friends of friends of friends of Alice."

In Relational SQL (Multiple JOINs):

Listing 4: Depth-3 Social Search in SQL

```sql
SELECT DISTINCT f3.name
FROM Users u
JOIN Friends f1 ON u.id = f1.user_id
JOIN Friends f2 ON f1.friend_id = f2.user_id
JOIN Friends f3 ON f2.friend_id = f3.user_id
WHERE u.name = 'Alice';
```

In Graph Cypher (Simple Pattern):

Listing 5: Depth-3 Social Search in Cypher

```cypher
MATCH (alice:Person {name: 'Alice'})-[:FRIENDS_WITH*3]-(f)
RETURN DISTINCT f.name;
```

As the query depth grows, SQL joins explode in complexity and memory cost (**Join Explosion**), while graph queries simply follow direct pointers.

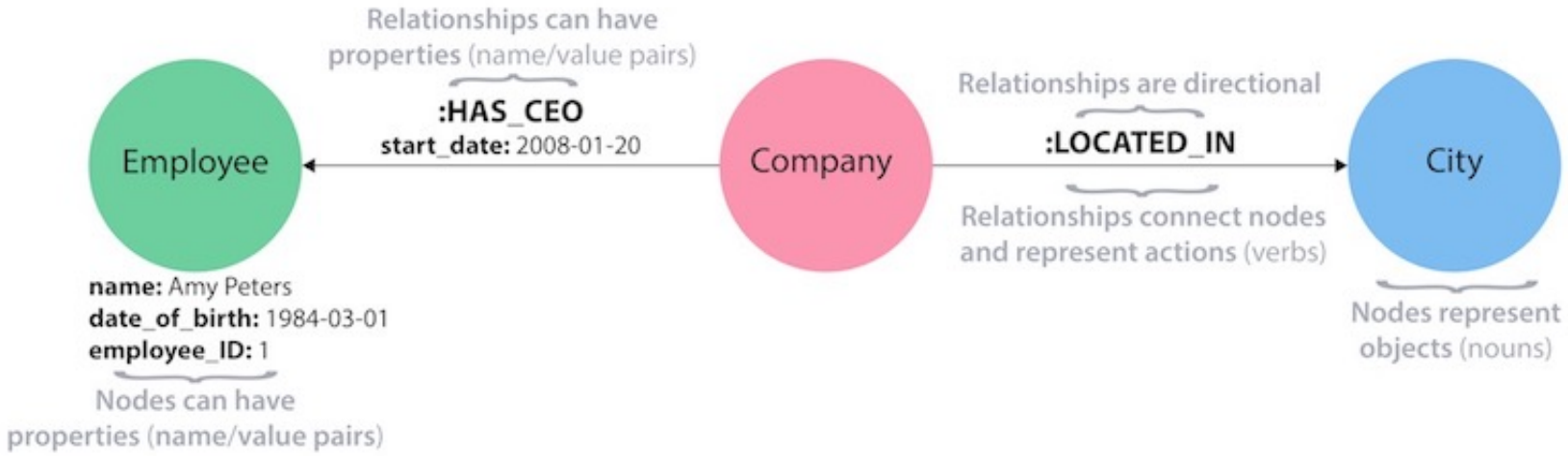


Figure 12: Property Graph Data Model Visualization

Source: Neo4j - Other Graph Database Technologies

1.6.2 The Property Graph Model

Most modern graph databases (like Neo4j) use the **Property Graph Model**, consisting of four core components:

1. **Nodes (Vertices):**
   Represent entities (e.g., `Person`, `Movie`, `Account`).

2. **Relationships (Edges):**
   Connect two nodes. Each relationship:

   - Has a **Direction** (start node → end node).
   - Has a **Type** written in uppercase (e.g., `:FRIENDS_WITH`, `:ACTED_IN`).

3. **Properties:**
   Key-value pairs stored on **both Nodes and Relationships**.

   - Node Property: `{name: 'Alice', age: 25}`
   - Relationship Property: `{since: 2021}`

4. **Labels:**
   Classify nodes into categories (e.g., `:Person`, `:Movie`).

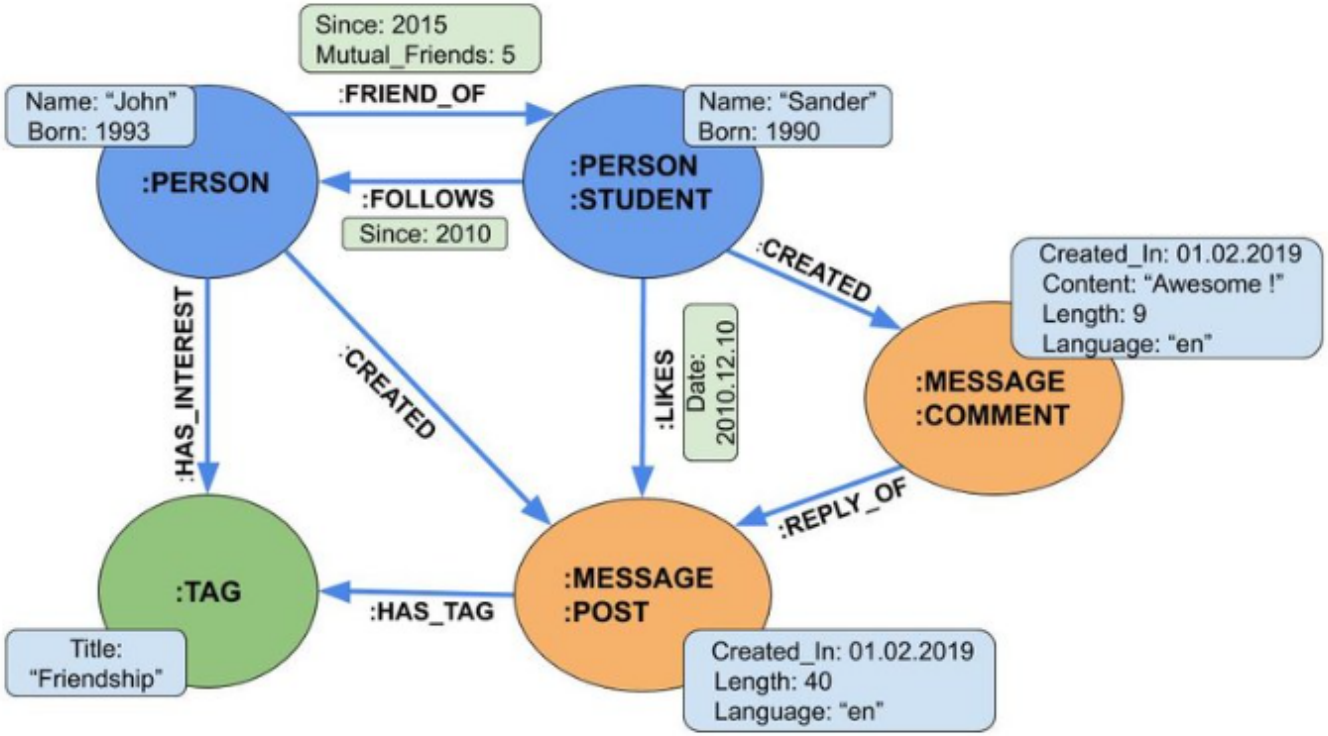


Figure 13: The Property Graph Model: Nodes, Directed Edges, Labels, and Properties

Source: Tale Project - Property Graphs

1.6.3 Visualizing a Property Graph

The figure below shows a social network represented as a property graph with directed connections, mutual friendships, and properties:

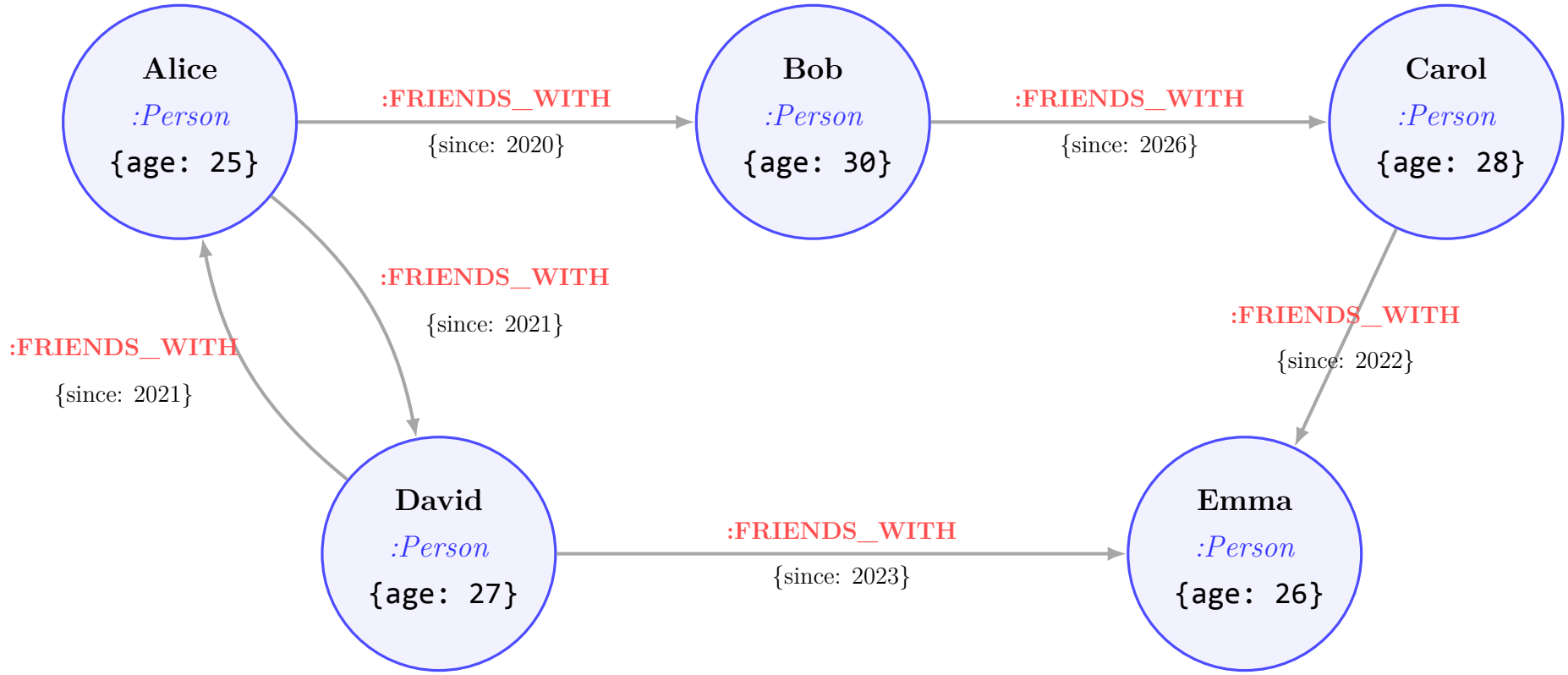


Figure 14: Property Graph: Directed Edges, Mutual Links, and Key-Value Properties

1.6.4 Index-Free Adjacency (The Key Advantage)

The core secret to graph database performance is **Index-Free Adjacency (IFA)**.

- **Index-Free:** The database does not search an index table to find connected data.
- **Adjacency:** Each node physically stores direct pointers to its neighbors.

Intuition

Imagine each person holds a personal list of their friends:

- In SQL, to find Alice's friends, you search a massive global `Friends` table.
- In a Graph DB, to find Alice's friends, you simply ask Alice! You don't scan the rest of the database.

Time Complexity: RDBMS vs. Graph Database

- **Relational Database (Index Lookups):** $O(\log N)$ per join hop, where N is the **total database size**. Traversal slows down as tables grow.
- **Graph Database (Index-Free Adjacency):** $O(k)$ per hop, where k is the **local degree** (only the direct neighbors of the node). Traversal speed is completely independent of total database size N!

1.6.5 Real-World Graph Use Cases

1. Financial Fraud Detection (Circular Money Transfers)

- Criminals route money through circular chains of accounts ($A \to B \to C \to A$) to hide its origin.
- Finding cycles in SQL requires slow recursive joins. In Cypher, it is a simple pattern:

```
MATCH (a:Account)-[:TRANSFERRED_TO]->(b)-[:TRANSFERRED_TO]->(c)
      -[:TRANSFERRED_TO]->(a)
RETURN a, b, c;
```

2. Recommendation Systems

- E-commerce: Find products purchased by other users who bought what you bought.

```
MATCH (me:User {name: 'John'})-[:BOUGHT]->(p)<-[:BOUGHT]-(other:User)
      -[:BOUGHT]->(rec:Product)
RETURN rec.title, count(other) AS score
ORDER BY score DESC LIMIT 5;
```

1.7 Graph Database Tool: Neo4j

Neo4j is the world's most popular native Graph Database.

Key Features:

- **Native Graph Storage:**
  Stores data as connected nodes and relationships on disk (Index-Free Adjacency), not as tables.
- **ACID Compliant:**
  Provides full transaction safety (Atomicity, Consistency, Isolation, Durability), making it suitable for enterprise finance and security.
- **Cypher Query Language:**
  The declarative query language created by Neo4j, designed specifically for pattern matching.
- **Rich Visualization Tools:**
  Includes **Neo4j Browser** for web-based exploration and **Neo4j Bloom** for visual analysis.

1.8 Graph Query Language: Cypher

Cypher is a declarative graph query language using intuitive **ASCII-art syntax** where query patterns visually resemble graph drawings:

```
(node:Label {properties}) -[:RELATIONSHIP]-> (target:Label)
```

- **Parentheses `()`** represent **Nodes** (circles).
- **Brackets `[]`** represent **Relationships** (arrows).
- **Arrows `-->`** represent **Direction**.
- **Colons `:`** represent **Labels** or **Types**.
- **Braces `{}`** represent **Properties** (key-value pairs).

Declarative ASCII-Art Philosophy

In SQL, you tell the database how to perform complex multi-table JOINs. In Cypher, you simply **draw the visual pattern** of the nodes and edges you want to find, and the engine traverses pointers directly.

1.8.1 Fundamental Query Patterns: Direction & Isolated Nodes

1. The Universal Pattern and Relationship Directions

Listing 6: Universal Query Pattern (Equivalent to `SELECT *` in SQL)

```
MATCH (a)-[b]-(c)
RETURN *;
```

Query Pattern	**Traversal Meaning**
`(a)-[b]->(c)`	**Outgoing relationships only** (directed from a to c)
`(a)<-[b]-(c)`	**Incoming relationships only** (directed from c to a)
`(a)-[b]-(c)`	**Either direction** (ignores edge direction / traverses either way)
`(a)-[*]-(c)`	**Paths with one or more relationships** (variable-length traversal)

Table 7: Cypher Relationship Patterns

2. Handling Isolated Nodes (`OPTIONAL MATCH`)

Listing 7: Listing Every Node and Relationship (Including Isolated Nodes)

```
MATCH (n)
OPTIONAL MATCH (n)-[r]->(m)
RETURN n, r, m;
```

- `MATCH (a)-[b]-(c)` matches only nodes connected by at least one relationship; isolated nodes are excluded.
- `OPTIONAL MATCH` acts like a SQL **LEFT JOIN**: returns all nodes n, filling r and m with `null` for isolated nodes.

1.8.2 1. CREATE and MERGE (Insert Data)

CREATE inserts new nodes and relationships:

Listing 8: Creating Nodes and Relationships

```
// Create a single node
CREATE (p:Person {name: 'Alice', age: 25});

// Create nodes and connect them
CREATE (bob:Person {name: 'Bob', age: 30}),
       (carol:Person {name: 'Carol', age: 28}),
       (bob)-[:FRIENDS_WITH {since: 2022}]->(carol);
```

MERGE creates only if the pattern does not already exist (upsert):

Listing 9: Idempotent Insertion with MERGE

```
MERGE (a:Person {name: 'Alice'})
MERGE (b:Person {name: 'Bob'})
MERGE (a)-[:FRIENDS_WITH]->(b);
```

1.8.3 2. MATCH and WHERE (Read and Filter Data)

MATCH finds patterns in the graph (equivalent to `SELECT` in SQL):

Listing 10: Pattern Matching and Filtering

```
// 1. Find all people
MATCH (p:Person)
RETURN p.name, p.age;

// 2. Find who works at Apple
MATCH (e:Person)-[:WORKS_AT]->(c:Company {name: 'Apple'})
RETURN e.name;

// 3. Filtering with WHERE
MATCH (p:Person)
WHERE p.age > 25 AND p.name STARTS WITH 'B'
RETURN p.name, p.age;

// 4. Friends of friends (2 hops)
MATCH (me:Person {name: 'Alice'})-[:FRIENDS_WITH]->(friend)-[:FRIENDS_WITH]->(fof)
RETURN fof.name;
```

1.8.4 3. SET and DELETE (Update and Remove Data)

SET (Update Properties and Labels):

Listing 11: Updating Nodes

```
MATCH (p:Person {name: 'Alice'})
SET p.age = 26, p:Manager
RETURN p;
```

DELETE and DETACH DELETE (Remove Data):

Listing 12: Deleting Nodes and Relationships

```
// Delete a relationship only
MATCH (a:Person {name: 'Alice'})-[r:FRIENDS_WITH]->(b:Person {name: 'Bob'})
DELETE r;

// Delete a node and all connected relationships
MATCH (p:Person {name: 'Alice'})
DETACH DELETE p;

// Delete everything in the database (Danger!)
MATCH (n)
DETACH DELETE n;
```

1.8.5 4. Advanced: Variable-Length Paths and Aggregation

Variable-Length Traversal (Hops):

Listing 13: Variable-Length Path Traversal

```
// Find connections between 1 and 3 hops away
MATCH (me:Person {name: 'Alice'})-[:FRIENDS_WITH*1..3]-(connection)
RETURN DISTINCT connection.name;
```

Shortest Path Algorithm:

Listing 14: Shortest Path Query

```
MATCH p = shortestPath(
    (a:Person {name: 'Alice'})-[*]-(b:Person {name: 'Kevin'})
)
RETURN p;
```

Aggregation (count):

Listing 15: Counting Friends

```
MATCH (p:Person {name: 'Alice'})-[:FRIENDS_WITH]->(friend)
RETURN count(friend) AS total_friends;
```

1.8.6 Using Neo4j with Python

To interact with Neo4j from Python, use the official `neo4j` driver:

```
pip install neo4j
```

Listing 16: Python Script Connecting to Neo4j

```
from neo4j import GraphDatabase

URI = "bolt://localhost:7687"
AUTH = ("neo4j", "password")

driver = GraphDatabase.driver(URI, auth=AUTH)

def add_friendship(tx, name1, name2):
    tx.run("""
        MERGE (a:Person {name: $name1})
        MERGE (b:Person {name: $name2})
        MERGE (a)-[:FRIENDS_WITH]->(b)
    """, name1=name1, name2=name2)

def get_friends(tx, name):
    result = tx.run("""
        MATCH (p:Person {name: $name})-[:FRIENDS_WITH]->(f)
        RETURN f.name AS friend
    """, name=name)
    return [record["friend"] for record in result]
```

Listing 17: Executing Graph Queries in Python

```
with driver.session() as session:
    # 1. Clear database
    session.run("MATCH (n) DETACH DELETE n")

    # 2. Insert data
    session.execute_write(add_friendship, "Alice", "Bob")
    session.execute_write(add_friendship, "Alice", "Carol")

    # 3. Query data
    friends = session.execute_read(get_friends, "Alice")
    print(f"Alice's friends: {friends}")

driver.close()
```

Terminal Output:

```
Alice's friends: ['Bob', 'Carol']
```

1.8.7 When NOT to Use Graph Databases

Workload	Why Graph DB is Suboptimal	Better Alternative
Heavy Global Aggregations	Calculating `AVG()` or `SUM()` across billions of rows	Relational (PostgreSQL)
Simple Key-Value Lookups	Point lookups don't need relationship pointers	Key-Value (Redis)
Pure Time-Series Logs	Storing high-speed timestamp streams doesn't need graph structure	Column-Family (Cassandra)
Standard Tabular Records	Simple CRUD without deep links is simpler in tables	Relational (PostgreSQL)

Table 8: When Graph Databases Are Not Recommended

Rule of Thumb

- If **relationships are secondary** $\longrightarrow$ Use Relational or Document.
- If **relationships are central** $\longrightarrow$ Use Graph.

1.9 Chapter Summary

- **Column-Family Stores (Cassandra):**
  Sparse sorted map: (Row Key, Column Family, Column Qualifier, Timestamp) $\rightarrow$ Value.
- **LSM Trees for Fast Writes:**
  Replaces B+ Trees with append-only sequential writes: writes hit in-memory **MemTable** and **WAL**, flush to immutable **SSTables**, and merge via **Compaction**.
- **Cassandra CQL Rules:**
  Design schemas for queries. **Partition Key** routes data to servers; **Clustering Key** sorts rows on disk. No JOINs.
- **Graph Databases (Neo4j):**
  Store nodes and directed relationships as first-class citizens. Relationships are direct physical pointers, avoiding expensive JOINs.
- **Index-Free Adjacency:**
  Query speed is $O(k)$ based on local neighborhood connections, completely independent of total database size.
- **Cypher Query Language:**
  Intuitive declarative ASCII-art pattern matching: `(p:Person)-[:ACTED_IN]->(m:Movie)`.
- **Polyglot Persistence:**
  No single database fits every workload; modern architectures combine Relational, Key-Value, Document, Column-Family, and Graph engines.

1.9.1 SQL and NoSQL Design choices: Multi-Database (Polyglot) Architecture

There is no one-size-fits-all strategy. Modern systems rarely use just one database. Instead, architects choose the right tool for each job.

- **PostgreSQL** for user accounts and financial transactions (ACID).
- **Redis** for ultra-fast session caching and rate-limiting.
- **MongoDB** for flexible product catalogs.
- **Cassandra** for high-speed sensor streams and activity logs.
- **Neo4j** for social graphs and recommendation engines.

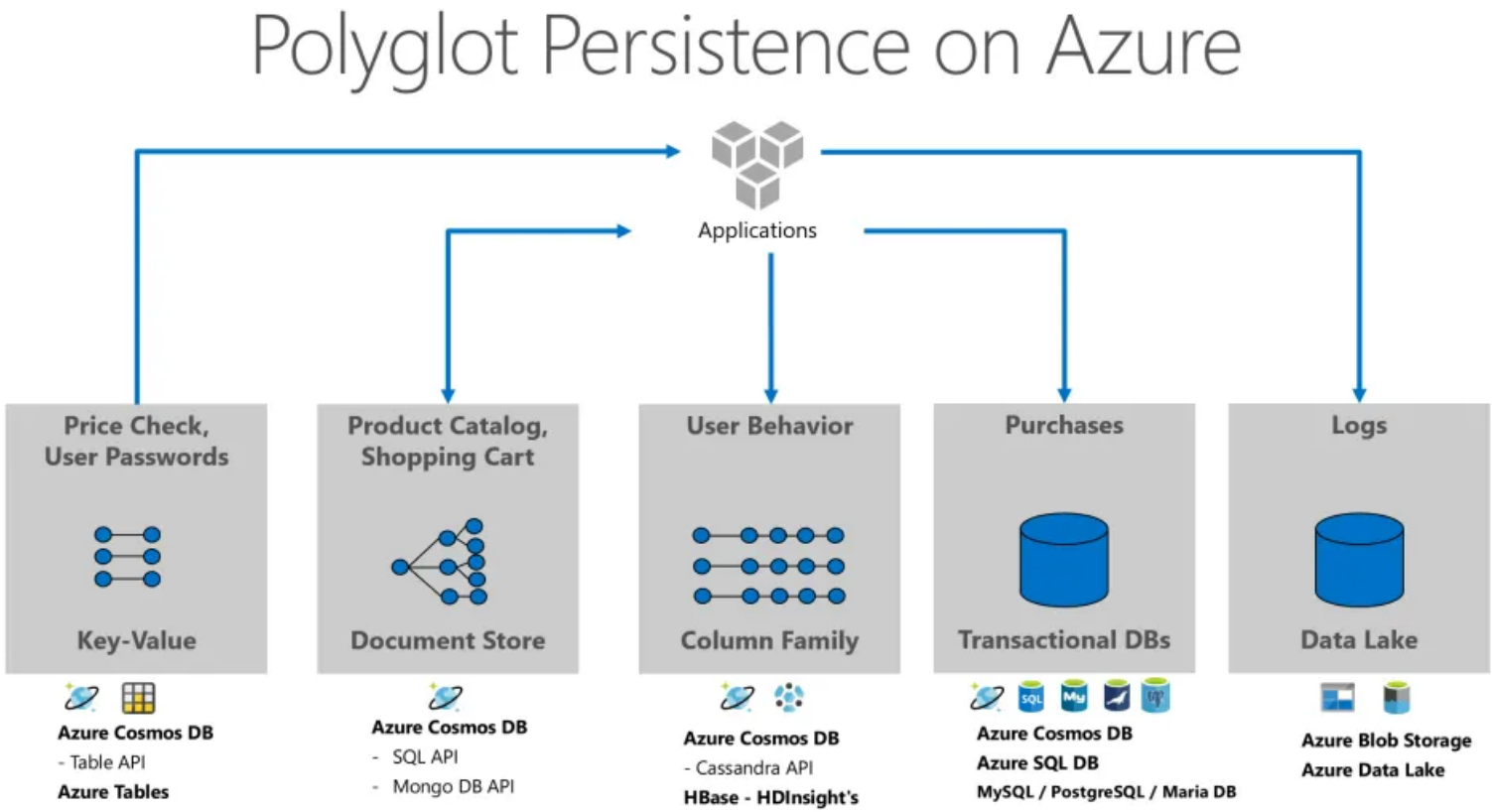


Figure 15: Polyglot Persistence in an E-Commerce Architecture

Source: Tuanhdotnet - Polyglot Persistence

1.9.2 SQL and NoSQL Databases Short Summary

Model	**Data Structure**	**Ideal Workload**	**Canonical Tools**
Relational	Tables & Rows	Structured transactions, ACID, reporting	PostgreSQL, MySQL
Key-Value	Key → Value	Caching, sessions, real-time counters	Redis, DynamoDB
Document	JSON / BSON	Flexible catalogs, semi-structured data	MongoDB
Column-Family	Multi-dim sorted map	Massive write streams, IoT telemetry	Apache Cassandra
Graph	Nodes & Edges	Highly connected networks, fraud, recommendations	Neo4j

Table 9: The Five Database Paradigms across Chapters 1–5

1.10 Activity/Homework: Graph Database Exploration (Neo4j)

1.10.1 Expected Learning Outcomes

After completing this activity, you will be able to:

- Run and access a local Neo4j database using Docker Compose.
- Explore a Property Graph containing labeled nodes, relationships, and properties.
- Write Cypher queries combining `MATCH`, `WHERE`, `RETURN`, and sorting directives.
- Traverse multi-hop paths and detect relationship patterns.
- Use aggregation functions such as `count()` to summarize graph data.

Objective: Practice writing Cypher queries on the built-in **Movie Graph Dataset** in Neo4j to retrieve, filter, aggregate, and analyze connected data.

1.10.2 Technology Stack

Docker & Docker Compose: Runs an isolated Neo4j container without local installation hassle.

Neo4j (Community Edition 5+): Graph Database running on ports `7474` (HTTP) and `7687` (Bolt protocol).

Neo4j Browser: Web UI at `http://localhost:7474` for running queries and visualizing results.

Python 3 (Optional): Can connect to Neo4j using the `neo4j` driver.

1.10.3 The Movie Graph Data Model & Schema

The lab uses the built-in **Movie Graph** dataset. The graph contains two primary Node Labels and six Relationship Types:

Entity	**Label / Type**	**Properties**
Node	`:Movie`	`title` (String), `released` (Int), `tagline` (String)
Node	`:Person`	`name` (String), `born` (Int)
Relationship	`:ACTED_IN`	`(:Person)-[:ACTED_IN {roles: [...]}]->(:Movie)`
Relationship	`:DIRECTED`	`(:Person)-[:DIRECTED]->(:Movie)`
Relationship	`:PRODUCED`	`(:Person)-[:PRODUCED]->(:Movie)`
Relationship	`:WROTE`	`(:Person)-[:WROTE]->(:Movie)`
Relationship	`:REVIEWED`	`(:Person)-[:REVIEWED {rating, summary}]->(:Movie)`
Relationship	`:FOLLOWS`	`(:Person)-[:FOLLOWS]->(:Person)`

Table 10: Movie Graph Schema Specification

1.10.4 Environment Setup Instructions

Built-in Movie Graph Database

By default, Neo4j includes the **Movie Graph** example database pre-packaged within the environment. No external data downloads, CSV imports, or SQL migrations are required.

Step 1: Create `docker-compose.yml`

Listing 18: Docker Compose Configuration

```
services:
  neo4j:
    image: neo4j:latest
    container_name: neo4j_lab
    ports:
      - "7474:7474"   # HTTP Browser
      - "7687:7687"   # Bolt Protocol
    environment:
      - NEO4J_AUTH=neo4j/password
```

Step 2: Start the Database

```
docker compose up -d
```

Step 3: Open Neo4j Browser

1. Open your browser and go to `http://localhost:7474`.
2. Connect with Username: `neo4j`, Password: `password`.

Step 4: Load the Movie Graph Dataset

1. In the Neo4j Browser top input bar, type:

```
:play movies
```

2. Go to Page 2 ("Write code").
3. Click the code block to load the sample data, and click the **Run** button.

Step 5: Verify Setup

```
MATCH (n) RETURN count(n) AS total_nodes;
```

If nodes appear ($\approx$ 170 nodes), the database is ready.

1.10.5 Student Tasks: 30 Cypher Queries

For each task:

- Write the Cypher query.
- Execute it in Neo4j Browser.
- Provide a 1–2 sentence explanation of what the query does.
- Provide a short summary of the results (e.g., number of rows returned).

Part A: Basic MATCH Queries (Queries 1–10)

1. Return all movies.
2. Return all people.
3. Return all actors (people with an `:ACTED_IN` relationship).
4. Find all movies released after 2000.
5. Find the movie titled “The Matrix”.
6. List all actors who acted in “The Matrix”.
7. Find all directors of “The Matrix”.

8. Return movies released between 1990 and 2000.
9. Find all movies Tom Hanks acted in.
10. Count the total number of movies.

Part B: Filtering & Conditions (Queries 11–20)

11. Find actors born after 1970.
12. Find movies with tagline containing "love" (case-insensitive).
13. Find movies with review rating greater than 50.
14. Return actors whose name starts with "K".
15. Find movies released before 1995.
16. Find people who both acted in and directed the same movie.
17. Find actors who acted in more than 2 movies.
18. Find movies with more than 3 actors.
19. Find the oldest actor.
20. Find the most recent movie.

Part C: Relationship Traversal (Queries 21–30)

21. Find co-actors of Tom Hanks.
22. Find people who worked with Keanu Reeves (acted, directed, or produced).
23. Find the shortest path between Keanu Reeves and Tom Hanks.
24. Find all people connected to Kevin Bacon within 2 hops.
25. Find actors who worked with the same director.
26. Find movies where two specific actors both acted.
27. Find directors who directed 3 or more movies.
28. Find actors who never directed a movie.
29. Find people who reviewed a movie they also acted in.
30. Find the top 3 most connected actors (most relationships).

1.10.6 Deliverables

Submit the following:

1. **Report Document (PDF):**
   Include all 30 numbered tasks with:
   - The Cypher query.
   - 1–2 sentence explanation.
   - Short result summary.
2. **Execution Screenshots:**
   Include screenshots from the Neo4j Browser demonstrating results for at least:
   - Query 6 (Actors in The Matrix).
   - Query 16 (People who acted and directed).
   - Query 23 (Shortest path between Keanu Reeves and Tom Hanks).
   - Query 30 (Top 3 most connected actors).
3. **Python Verification Script (Optional / Bonus):**
   A short script using the official `neo4j` library demonstrating connection and execution of any 2–3 queries.